

HƯỚNG DẪN KIẾN TRÚC & TỔ CHỨC SOURCE CODE

Dự án: EduTalk HUIT — Hệ thống AI Tư vấn Hướng nghiệp & Tuyển sinh

Trường Đại học Công Thương TP.HCM (HUIT) Nhóm Nghiên cứu K14

Tài liệu này dành cho tất cả thành viên trong nhóm. Hãy đọc kỹ trước khi bắt tay vào code bất cứ tính năng nào.

Mục lục

1. Tổng quan dự án
 2. Kiến trúc hệ thống (System Architecture)
 3. Cấu trúc Monorepo
 4. Backend — FastAPI (Python)
 5. Web Frontend — Next.js (TypeScript)
 6. Mobile App — Flutter (Dart)
 7. Luồng xác thực (Authentication Flow)
 8. Cơ sở dữ liệu & Dịch vụ bên ngoài
 9. CI/CD & Kiểm tra chất lượng code
 10. Hướng dẫn thêm tính năng mới (từ A-Z)
 11. Quy tắc làm việc nhóm
 12. Cách cài đặt & chạy dự án
-

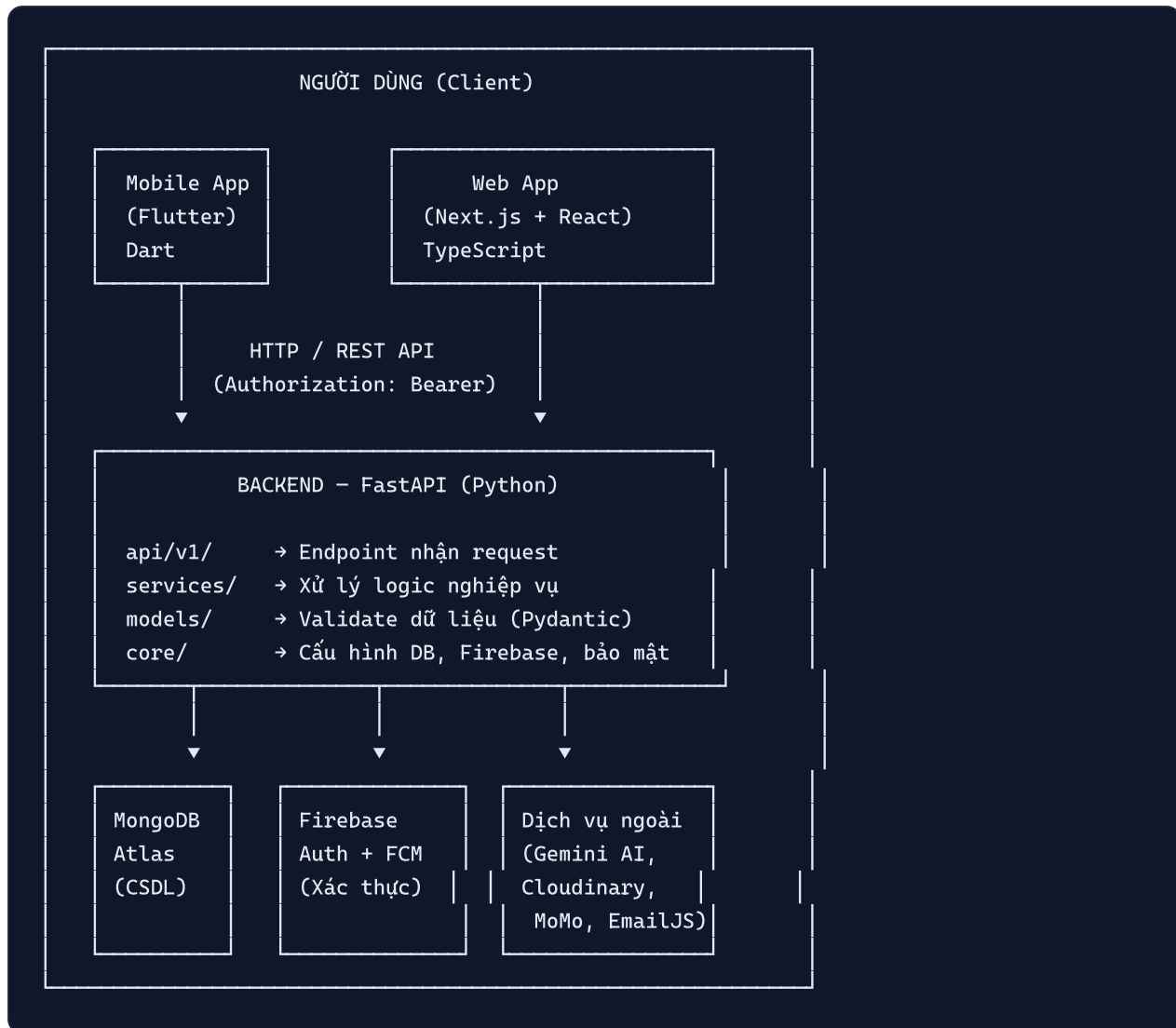
1. Tổng quan dự án

EduTalk là một hệ thống tư vấn hướng nghiệp & tuyển sinh thông minh dành cho học sinh THPT, sử dụng:

- **AI Chatbot (Gemini):** Trả lời câu hỏi về ngành học, trường đại học, điểm chuẩn.
- **Dự đoán ngành phù hợp (XGBoost + Cosine Similarity):** Nhập điểm thi → Gợi ý ngành học phù hợp nhất tại HUIT.
- **Cộng đồng (Community):** Diễn đàn thảo luận kiểu mạng xã hội (bài viết, thích, bình luận).
- **Tin tức tuyển sinh (Crawler):** Tự động cào và tóm tắt tin tuyển sinh bằng AI.
- **Admin Dashboard:** Quản lý người dùng, bài viết, doanh thu Premium, duyệt tin.

Hệ thống chạy trên **3 nền tảng**: Web (Next.js), Mobile (Flutter) và Backend API (FastAPI), tất cả đều nằm trong **một Monorepo duy nhất**.

2. Kiến trúc hệ thống



Điểm mấu chốt cần hiểu:

- Mobile và Web **KHÔNG** giao tiếp trực tiếp với Database.
- Mọi thao tác đều đi qua **Backend FastAPI** thông qua REST API.
- Firebase **chỉ** dùng để xác thực (Auth) và gửi thông báo (FCM). Dữ liệu chính lưu ở **MongoDB Atlas**.
- Backend deploy trên **Render** tại: <https://edutalk-7ndf.onrender.com>

3. Cấu trúc Monorepo

Khi clone repo về, bạn sẽ thấy cấu trúc sau:

Thesis-EduTalk/	← Thư mục gốc (Monorepo)
backend/	← ● Server API (FastAPI + Python)
app/	← Mã nguồn chính
main.py	← Khởi tạo FastAPI, CORS, mount router
api/v1/	← Các file định nghĩa API endpoint
services/	← Logic nghiệp vụ (AI, Auth, CRUD)
models/	← Schema validate dữ liệu (Pydantic)
core/	← Cấu hình DB, Firebase
db/seed/	← Dữ liệu mẫu (seed data)
requirements.txt	← Danh sách thư viện Python
Dockerfile	← Đóng gói Docker cho deploy
.env.example	← Mẫu biến môi trường (copy → .env)
serviceAccountKey.json	←  Key Firebase (KHÔNG push lên Git!)
web/	← ● Giao diện Web (Next.js + React)
src/	
app/	← Routing (App Router)
layout.tsx	← Layout gốc (metadata, font)
globals.css	← Style toàn cục
(main)/	← Nhóm route người dùng (có Navbar)
page.tsx	← Trang chủ
chat/	← Trang chat AI
community/	← Trang cộng đồng
predict/	← Trang dự đoán ngành
news/	← Trang tin tức tuyển sinh
profile/	← Trang cá nhân
settings/	← Trang cài đặt
auth/	← Đăng nhập / Đăng ký
...	
dashboard/	← Nhóm route Admin (layout riêng)
layout.tsx	← Sidebar admin
page.tsx	← Dashboard tổng quan
news/	← Duyệt tin tức
components/	← Các component UI tái sử dụng
layout/	← Navbar, BottomNav, Footer
features/	← Component theo tính năng (ChatWidget)
ui/	← Component chung (Button, Input, Modal)
shared/	← Component dùng chung (Avatar, Badge)
services/	← Lớp gọi API (gói Axios thành hàm)
auth.ts	← AuthService: login, register, googleLogin
store/	← State toàn cục (Zustand)
useAuthStore.ts	← Lưu user đang đăng nhập
lib/	← Tiện ích, cấu hình
api.ts	← Axios instance (tự đính Token)
firebase.ts	← Khởi tạo Firebase Web SDK
hooks/	← Custom React Hooks (tạo sẵn)
types/	← TypeScript types & interfaces (tạo sẵn)
schemas/	← Validate form bằng Zod (tạo sẵn)
config/	← Cấu hình ứng dụng (tạo sẵn)
package.json	

mobile/	←  Ứng dụng Mobile (Flutter)
lib/	
main.dart	← Entry point, khởi tạo Firebase
screens/	← Các màn hình UI
services/	← Lớp gọi API (dùng package http)
api_client.dart	← HTTP Client chung (tự đính token)
models/	← Data class (parse JSON → Object)
widgets/	← Widget UI tái sử dụng
provider/	← State management (Provider)
admin/	← Tính năng dành cho Admin
pubspec.yaml	← Danh sách thư viện Flutter
docs/	←  Tài liệu dự án
scripts/	←  Script tự động (generate API client)
.github/workflows/ci.yml	←  CI/CD (GitHub Actions)
docker-compose.yml	←  Chạy bằng Docker
.gitignore	← Danh sách file không push lên Git

4. Backend — FastAPI (Python)

4.1. Kiến trúc 3 lớp

Backend tổ chức theo mô hình **Controller** → **Service** → **Database**:



4.2. Danh sách API Routers (`api/v1/`)

File	Prefix	Chức năng
<code>auth.py</code>	<code>/api/v1/auth</code>	Đăng ký, đăng nhập (email + Google), đổi mật khẩu
<code>users.py</code>	<code>/api/v1/users</code>	CRUD thông tin người dùng, cập nhật avatar
<code>predict.py</code>	<code>/api/v1/predict</code>	Dự đoán ngành học (XGBoost + Cosine Similarity)
<code>majors.py</code>	<code>/api/v1/majors</code>	Lấy danh sách ngành HUIT
<code>survey.py</code>	<code>/api/v1/survey</code>	Khảo sát trắc nghiệm hướng nghiệp
<code>chat.py</code>	<code>/api/v1/chat</code>	Chat AI với Gemini, lịch sử chat
<code>posts.py</code>	<code>/api/v1/posts</code>	Bài viết cộng đồng (CRUD, like, comment, upload ảnh)
<code>support.py</code>	<code>/api/v1/support</code>	Gửi yêu cầu hỗ trợ
<code>news.py</code>	<code>/api/v1/news</code>	Lấy danh sách tin tức tuyển sinh
<code>admin.py</code>	<code>/api/v1/admin</code>	Dashboard admin: quản lý user, posts, doanh thu
<code>admin_news.py</code>	<code>/api/v1/admin/news</code>	Duyệt / từ chối tin tức

4.3. Danh sách Services (`services/`)

File	Chức năng
<code>auth_service.py</code>	Đăng ký, đăng nhập, Google Sign-in, OTP, đổi mật khẩu
<code>gemini_service.py</code>	Giao tiếp với Google Gemini AI, phân tích xu hướng ngành
<code>predict_service.py</code>	Thuật toán dự đoán ngành (39 ngành HUIT, 15 tổ hợp)
<code>post_service.py</code>	CRUD bài viết, like, comment, báo cáo bài viết
<code>news_service.py</code>	Quản lý tin tức (lấy danh sách, duyệt, từ chối)
<code>crawler_service.py</code>	Tự động cào tin tuyển sinh + tóm tắt bằng Gemini AI

4.4. Models (Pydantic Schemas) (`models/`)

Mỗi file tương ứng với một nhóm API và định nghĩa cấu trúc dữ liệu đầu vào (Request Body):

File	Ví dụ
<code>auth_models.py</code>	<code>LoginRequest(email, password)</code>
<code>user_models.py</code>	<code>UpdateProfileRequest(name, phone, avatar)</code>
<code>post_models.py</code>	<code>CreatePostRequest(content, imageUrl, ...)</code>
<code>predict_models.py</code>	<code>PredictRequest(scores: dict)</code>
<code>chat_models.py</code>	<code>ChatRequest(message, history)</code>
<code>survey_models.py</code>	<code>SurveySubmitRequest(answers: list)</code>
<code>support_models.py</code>	<code>SupportTicket(subject, message)</code>
<code>admin_models.py</code>	<code>UpdateUserRoleRequest(userId, role)</code>

5. Web Frontend — Next.js (TypeScript)

5.1. Công nghệ sử dụng

Công nghệ	Vai trò
Next.js 16	Framework React, App Router
React 19	Thư viện UI
TypeScript	Ngôn ngữ (có kiểm tra kiểu)
TailwindCSS 4	Framework CSS utility-first
Zustand	State management (nhẹ hơn Redux)
Axios	HTTP client (gọi API)
Firebase Web SDK	Đăng nhập Google trên Web
Framer Motion	Hiệu ứng animation
Lucide React	Bộ icon SVG
Recharts	Biểu đồ cho Dashboard
Lottie React	Animation Lottie
React Markdown	Render Markdown (cho câu trả lời AI)

5.2. Hệ thống Routing (App Router)

Next.js App Router sử dụng **thư mục = route**:

```

app/
├── layout.tsx          ← Root Layout (font Inter, metadata SEO)
├── globals.css         ← CSS toàn cục (TailwindCSS + animation)
├── (main)/             ← Route Group cho NGƯỜI DÙNG (có Navbar + Footer)
│   ├── layout.tsx      ← Layout chung: Navbar trên, Footer dưới, ChatWidget
│   ├── page.tsx        ← "/" – Trang chủ
│   ├── auth/
│   │   ├── login/page.tsx ← "/auth/login" – Đăng nhập
│   │   └── register/     ← "/auth/register" – Đăng ký
│   ├── chat/page.tsx   ← "/chat" – Chat với AI
│   ├── community/page.tsx ← "/community" – Cộng đồng bài viết
│   ├── news/page.tsx   ← "/news" – Danh sách tin tức
│   ├── news/[id]/page.tsx ← "/news/123" – Chi tiết 1 bài tin
│   ├── predict/page.tsx ← "/predict" – Nhập điểm, dự đoán ngành
│   ├── result/page.tsx  ← "/result" – Kết quả dự đoán
│   ├── majors/page.tsx  ← "/majors" – Danh sách ngành HUIT
│   ├── profile/page.tsx ← "/profile" – Trang cá nhân
│   ├── history/page.tsx ← "/history" – Lịch sử chat
│   └── settings/page.tsx ← "/settings" – Cài đặt
├── dashboard/         ← Route Group cho ADMIN (layout Sidebar riêng)
│   ├── layout.tsx      ← Sidebar admin (dark theme)
│   ├── page.tsx        ← "/dashboard" – Tổng quan (biểu đồ, thống kê)
│   └── news/page.tsx   ← "/dashboard/news" – Duyệt tin tức

```

Lưu ý: Dấu ngoặc đơn **(main)** là **Route Group** — nó chỉ dùng để nhóm các trang dùng chung Layout, KHÔNG xuất hiện trên URL. Ví dụ: **(main)/chat/page.tsx** → URL là **/chat** chứ KHÔNG phải **/main/chat**.

5.3. Thư mục `components/`

Đã được chia nhỏ theo chuẩn Best Practice:

```

components/
├── layout/             ← Navbar, BottomNav (cấu trúc giao diện chung)
├── features/
│   └── chat/           ← ChatWidget.tsx (widget chat popup)
├── ui/                ← Component UI cơ bản (Button, Input, Modal) ← Tạo sẵn
└── shared/            ← Component dùng lại nhiều nơi (Avatar, Badge) ← Tạo sẵn

```

5.4. Luồng gọi API từ Web

```

Trang page.tsx
├──
│   import { AuthService } from "@services/auth"
│   const data = await AuthService.login(email, password)
│
└──
    services/auth.ts
    ├──
    │   import api from "@lib/api"
    │   api.post("/api/v1/auth/login", { email, password })
    │
    └──
        lib/api.ts (Axios Instance)
        ├──
        │   Interceptor tự động đính kèm Token vào header
        │   Authorization: Bearer <token>
        │
        └──
            Backend FastAPI → MongoDB → Response JSON
  
```

Quy tắc quan trọng: Khi cần gọi API mới, **KHÔNG** viết `axios.get(...)` trực tiếp trong `page.tsx`. Hãy tạo hàm trong `services/` rồi import vào page.

5.5. State Management (Zustand)

File `store/useAuthStore.ts` lưu trữ thông tin user đang đăng nhập:

```

// Sử dụng trong bất kỳ component nào:
import { useAuthStore } from "@store/useAuthStore";

const user = useAuthStore((state) => state.user); // Lấy user
const logout = useAuthStore((state) => state.logout); // Đăng xuất
  
```

Zustand dùng middleware `persist` → tự động lưu vào `localStorage` → user refresh trang không bị mất trạng thái đăng nhập.

6. Mobile App — Flutter (Dart)

6.1. Cấu trúc `lib/`

<code>lib/</code>	
<code>main.dart</code>	← Entry point, khởi tạo Firebase, định nghĩa routes
<code>firebase_options.dart</code>	← Config Firebase tự sinh bởi FlutterFire CLI
<code>screens/</code>	← Tất cả các màn hình UI
<code>home.dart</code>	← Trang chủ (bottom navigation)
<code>home_page.dart</code>	← Nội dung trang Home
<code>Login.dart</code>	← Đăng nhập (Email + Google)
<code>Register.dart</code>	← Đăng ký
<code>Profile.dart</code>	← Trang cá nhân
<code>ThaoLuan.dart</code>	← Cộng đồng / Diễn đàn
<code>DuLieu.dart</code>	← Nhập dữ liệu điểm thi
<code>PhanTich.dart</code>	← Phân tích kết quả
<code>KetQua.dart</code>	← Hiển thị kết quả dự đoán
<code>LichSu.dart</code>	← Lịch sử dự đoán
<code>ai_chat_screen.dart</code>	← Chat AI
<code>Premium_screen.dart</code>	← Nâng cấp Premium
<code>Setting.dart</code>	← Cài đặt
<code>support_screen.dart</code>	← Hỗ trợ
<code>admin/</code>	← Màn hình admin (trên mobile)
<code>services/</code>	← Lớp giao tiếp API
<code>api_client.dart</code>	← HTTP Client chung (GET, POST, PUT, DELETE)
<code>auth_service.dart</code>	← Đăng nhập, đăng ký, Google Sign-in
<code>ai_chat_service.dart</code>	← Gọi API chat Gemini
<code>post_service.dart</code>	← CRUD bài viết cộng đồng
<code>admin_service.dart</code>	← API quản trị
<code>payment_service.dart</code>	← Thanh toán MoMo
<code>...</code>	
<code>models/</code>	← Data class
<code>user_model.dart</code>	← UserModel (parse JSON → Object)
<code>post_model.dart</code>	← PostModel
<code>payment_model.dart</code>	← PaymentModel
<code>widgets/</code>	← Widget tái sử dụng
<code>premium_badge.dart</code>	← Badge hiển thị Premium
<code>premium_upgrade_dialog.dart</code>	
<code>bank_transfer_sheet.dart</code>	
<code>provider/</code>	
<code>Themenotifier.dart</code>	← Quản lý Light/Dark theme

6.2. Luồng gọi API từ Mobile

```
// Trong Screen, gọi hàm từ Service:
final result = await AuthService.login(email, password);

// Service gọi ApiClient:
class AuthService {
  Future<Map> login(email, password) {
    return ApiClient.post('/api/v1/auth/login', body: { ... });
  }
}

// ApiClient tự đính token Firebase:
class ApiClient {
  static const baseUrl = 'https://edutalk-7ndf.onrender.com';
  // Tự lấy FirebaseAuth.instance.currentUser.getIdToken()
}
```

7. Luồng xác thực (Authentication Flow)

7.1. Đăng nhập Email/Password

```

[Mobile/Web] → POST /api/v1/auth/login { email, password }
      ↓
[Backend] auth_service.py
      |   Gọi Firebase REST API signInWithPassword
      |   Nhận lại idToken + refreshToken
      ↓
[Backend] Trả về { token, user: { id, name, role, isPremium } }
      ↓
[Mobile/Web] → Lưu token vào LocalStorage / FirebaseAuth
              → Lưu user info vào Zustand Store (Web) hoặc Provider (Mobile)
  
```

7.2. Đăng nhập Google

```

[Mobile/Web] → Firebase Web SDK / Google Sign-In
              → Nhận idToken từ Google
              → POST /api/v1/auth/google-login { idToken }
      ↓
[Backend] Firebase Admin verify idToken
      |   Nếu user chưa có → tạo mới trong MongoDB
      |   Trả về { token, user }
      ↓
[Mobile/Web] → Lưu token & user (giống flow trên)
  
```

7.3. Cách Token được đính kèm tự động

Trên Web (`lib/api.ts`):

- Axios Interceptor tự lấy token từ `Firebase Web SDK` hoặc `localStorage` .
- Mỗi request gửi đi đều có header: `Authorization: Bearer <token>` .

Trên Mobile (`services/api_client.dart`):

- `ApiClient` tự lấy token từ `FirebaseAuth.instance.currentUser.getIdToken()` .
- Header tương tự: `Authorization: Bearer <token>` .

Trên Backend (`api/v1/*.py`):

- Đọc header `Authorization` , tách lấy token.
 - Gọi `Firebase Admin SDK` verify token → lấy `uid` .
 - Dùng `uid` để truy vấn user trong MongoDB.
-

8. Cơ sở dữ liệu & Dịch vụ bên ngoài

8.1. MongoDB Atlas (Cơ sở dữ liệu chính)

Kết nối qua biến `MONGO_URI` trong file `.env`. Driver: `motor` (async).

Collection	Mô tả
<code>users</code>	Thông tin người dùng
<code>posts</code>	Bài viết cộng đồng
<code>comments</code>	Bình luận
<code>chat_history</code>	Lịch sử chat AI
<code>predictions</code>	Lịch sử dự đoán ngành
<code>surveys</code>	Kết quả khảo sát hướng nghiệp
<code>news</code>	Tin tức tuyển sinh (crawler)
<code>transactions</code>	Giao dịch thanh toán Premium
<code>support_tickets</code>	Yêu cầu hỗ trợ
<code>admin_notifications</code>	Thông báo cho admin

8.2. Firebase (Xác thực & Thông báo)

Dịch vụ	Vai trò
Firebase Authentication	Quản lý tài khoản (Email + Google Sign-in)
Firebase Cloud Messaging	Gửi push notification đến Mobile
Firebase App Check	Bảo vệ API khỏi request giả mạo

8.3. Dịch vụ bên ngoài khác

Dịch vụ	Biến <code>.env</code>	Vai trò
Google Gemini	<code>GEMINI_API_KEY</code>	AI Chatbot + Tóm tắt tin tức
Cloudinary	<code>CLOUDINARY_CLOUD_NAME/KEY/SECRET</code>	Upload ảnh bài viết
MoMo	<code>MOMO_PARTNER_CODE/ACCESS/SECRET</code>	Thanh toán Premium
EmailJS	<code>EMAILJS_SERVICE_ID/ ...</code>	Gửi OTP xác thực email

9. CI/CD & Kiểm tra chất lượng code

Dự án sử dụng **GitHub Actions** (file `.github/workflows/ci.yml`). Mỗi khi push lên `main` hoặc tạo Pull Request, CI sẽ tự động chạy:

Backend Lint

```
# Kiểm tra code style Python:
ruff check backend/app      # Phát hiện lỗi logic, import thừa, bảo mật
black --check backend/app   # Kiểm tra format code (khoảng trắng, dấu phẩy)
```

Web Lint

```
# Kiểm tra code style TypeScript/React:
cd web && npm run lint      # ESLint kiểm tra hook rules, unused vars, img tags
```

Quy tắc: Trước khi push code, hãy chạy lint ở local để tránh CI fail:

```
# Backend:
ruff check backend/app
black backend/app

# Web:
cd web && npm run lint
```

10. Hướng dẫn thêm tính năng mới (từ A-Z)

Ví dụ thực tế: Thêm tính năng "Đánh giá trường đại học" (University Review).

Bước 1: Backend — Tạo Model

```
# backend/app/models/review_models.py
from pydantic import BaseModel

class CreateReviewRequest(BaseModel):
    university_name: str
    rating: int          # 1-5 sao
    content: str
```

Bước 2: Backend — Tạo Service

```
# backend/app/services/review_service.py
from app.core.mongodb import get_db
from datetime import datetime, timezone

class ReviewService:
    async def create_review(self, uid: str, data: dict) → str:
        db = get_db()
        result = await db["reviews"].insert_one({
            "userId": uid,
            "universityName": data["university_name"],
            "rating": data["rating"],
            "content": data["content"],
            "createdAt": datetime.now(timezone.utc),
        })
        return str(result.inserted_id)
```

Bước 3: Backend — Tạo API Route

```
# backend/app/api/v1/reviews.py
from fastapi import APIRouter, Header
from app.services.review_service import ReviewService
from app.models.review_models import CreateReviewRequest

router = APIRouter()
review_service = ReviewService()

@router.post("/create")
async def create_review(body: CreateReviewRequest, authorization: str = Header(...)):
    # verify token, lấy uid...
    review_id = await review_service.create_review(uid, body.dict())
    return {"status": "success", "reviewId": review_id}
```

Bước 4: Backend — Đăng ký Router

```
# backend/app/main.py - Thêm dòng:
from app.api.v1 import reviews
app.include_router(reviews.router, prefix="/api/v1/reviews", tags=["Reviews"])
```

Bước 5: Web — Tạo Service gọi API

```
// web/src/services/review.ts
import api from "@/lib/api";

export const ReviewService = {
  create: async (universityName: string, rating: number, content: string) => {
    const res = await api.post("/api/v1/reviews/create", {
      university_name: universityName,
      rating,
      content,
    });
    return res.data;
  },
};
```

Bước 6: Web — Tạo trang UI

```
web/src/app/(main)/reviews/page.tsx ← URL: /reviews
```

Bước 7: Mobile — Tạo Service + Screen tương ứng

```
// mobile/lib/services/review_service.dart  
// mobile/lib/screens/review_screen.dart
```

11. Quy tắc làm việc nhóm

11.1. Quy tắc Git

- **Branch chính:** `main` — luôn ở trạng thái ổn định, CI pass.
- **Commit message:** Dùng tiếng Việt hoặc tiếng Anh, bắt đầu bằng prefix:
 - `feat:` — tính năng mới
 - `fix:` — sửa bug
 - `style:` — format code (không thay đổi logic)
 - `refactor:` — tái cấu trúc code
 - `docs:` — cập nhật tài liệu
 - `chore:` — việc lặt vặt (cập nhật `.gitignore`, xóa file thừa)

11.2. Quy tắc code

Quy tắc	Áp dụng cho
Không dùng <code>any</code> trong TypeScript	Web
Không hard-code dữ liệu fake	Toàn bộ
Luôn khai báo kiểu (Pydantic / TypeScript)	Backend + Web
Mỗi service 1 file riêng	Backend + Web
Không gọi API trực tiếp trong page/screen	Web + Mobile
Format code trước khi push (black / eslint)	Toàn bộ

11.3. Phân công theo chiều dọc (Vertical Slice)

Thay vì chia "người A làm Frontend, người B làm Backend", hãy chia theo **tính năng**:

Người A: Tính năng "Đánh giá trường"

- Viết `model` + `service` + `route` Backend
- Viết `service` + `page` Web
- Viết `service` + `screen` Mobile

Người B: Tính năng "Chat nhóm"

- Viết `model` + `service` + `route` Backend
- Viết `service` + `page` Web
- Viết `service` + `screen` Mobile

Cách này giúp mỗi người hiểu trọn vẹn 1 tính năng từ đầu đến cuối.

12. Cách cài đặt & chạy dự án

12.1. Yêu cầu

- **Python** ≥ 3.10 (khuyến nghị dùng Conda hoặc venv)
- **Node.js** ≥ 18
- **Flutter SDK** ≥ 3.10
- File `backend/serviceAccountKey.json` (lấy từ Firebase Console)
- File `backend/.env` (copy từ `.env.example` và điền thông tin thật)

12.2. Backend

```
cd backend
python -m venv venv

# Windows PowerShell:
.\venv\Scripts\Activate.ps1

# Mac/Linux:
source venv/bin/activate

pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Server sẽ chạy tại: `http://localhost:8000` Xem API docs tại: `http://localhost:8000/docs` (Swagger UI tự động)

12.3. Web

```
cd web
npm install
npm run dev
```

Web sẽ chạy tại: `http://localhost:3000`

12.4. Mobile

```
cd mobile  
flutter pub get  
flutter run
```

12.5. Docker (chạy nhanh toàn bộ)

```
docker-compose up --build
```

Lệnh trên sẽ khởi động Backend (port 8000) + Web (port 3000) + PostgreSQL (port 5432).

Lưu ý: Docker compose hiện cấu hình sẵn PostgreSQL nhưng Backend thực tế đang dùng MongoDB Atlas. File `docker-compose.yml` có thể cần cập nhật nếu muốn chạy hoàn toàn offline.

Tài liệu này được tạo ngày 10/08/2026